Day 1 - HTML Basics: Document Structure, Common Elements, Attributes

Day 1 — HTML Basics: Document Structure, Common Elements, Attributes

If you haven't read `day00-primer/lesson.md` yet, read that first — it covers what a tag, element, and attribute are, and the minimal shape of an HTML document, which everything below assumes you already know.

Objectives

- Get comfortable with the full basic document skeleton and why each part exists
- Meet the handful of elements you'll use in nearly every single page you ever build: headings, paragraphs, links, images, and lists
- Understand attributes in more depth, including ones that matter for accessibility (a theme you'll return to repeatedly)
- Understand *why* choosing the semantically correct element matters, not just picking whichever one happens to look right

Headings — `<h1>` through `<h6>`

HTML gives you six levels of heading, from most important (`<h1>`) to least important (`<h6>`):

```
<h1>Page Title</h1>
<h2>A Major Section</h2>
<h3>A Subsection</h3>
```

By default, browsers render `<h1>` the largest and boldest, shrinking progressively down to `<h6>`. It's tempting, as a beginner, to just pick whichever heading level happens to produce the font size you want visually — **resist this.** Heading levels are meant to describe the actual *outline* of your document's content, similar to how a book has chapters, then sections, then subsections — a screen reader (software that reads a page aloud for a visually impaired user) and search engines both rely on this structure to understand your page, completely independent of how large the text visually appears. If you want a heading to merely *look* smaller without changing its structural importance, you'll learn the correct tool for that on Day 3 (CSS) — don't misuse heading levels to fake a visual effect.

Use exactly one `<h1>` **per page**, representing the single main topic of that page, and nest the rest logically beneath it (don't jump from `<h1>` straight to `<h4>`, skipping levels).

Paragraphs — `<p>`

```
<p>This is a paragraph of text. It can contain several sentences,
and the browser will automatically wrap the lines to fit the available width.</p>
```

Notice something important: the line breaks *inside* your HTML source code (where you happened to press Enter while typing) have **no effect** on how the paragraph is displayed — the browser collapses any amount of whitespace (spaces, tabs, line breaks) in your source into a single space when rendering text. The paragraph will visually wrap based on the width of its container on screen, not based on where you happened to break lines in your source file. This surprises many beginners the first time they see it — go ahead and try adding extra line breaks and spaces inside a `<p>` in today's exercise, reload, and confirm the rendered result doesn't change.

Links — `<a>`

You already saw this on Day 0: `<a>` (short for "anchor") creates a hyperlink.

```
<a href="https://example.com">Visit Example</a>
<a href="about.html">About This Site</a>
<a href="#section2">Jump to Section 2</a>
<a href="mailto:someone@example.com">Email Us</a>
```

The `href` attribute (short for "hypertext reference") is what actually specifies the destination — the visible, clickable text between `<a>` and `</a>` can be completely different from the destination itself. A few different kinds of destinations worth knowing:

- A full web address (`https://example.com`) — an **absolute** link, pointing to a location anywhere on the internet.
- A filename or path relative to the current page (`about.html`) — a **relative** link, pointing to another page in the same project, without needing the full address.
- A `#` followed by an element's `id` attribute (explained below) — jumps to a specific spot *within the same page*, rather than loading a different page at all.
- `mailto:` — opens the visitor's email program with a new message addressed to that address, rather than navigating anywhere.

Images — `<img>`

```
<img src="dog.jpg" alt="A golden retriever sitting in a park">
```

`<img>` is a **self-closing** element (recall from Day 0 — no separate closing tag, since it doesn't wrap around any content). Two attributes matter enormously:

- `src` (source) — the file path or web address of the actual image to display.
- `alt` (alternative text) — a plain-text description of the image's content, displayed if the image fails to load, and — much more importantly — read aloud by screen readers for visually impaired users, since they have no way to actually see the image itself. **Always write a meaningful `alt`, describing what the image actually shows** — never leave it out, and never just repeat the filename. If an image is purely decorative and conveys no real information (a background flourish, say), the correct practice is `alt=""` (present, but deliberately empty) — this explicitly tells screen readers "skip this, there's nothing meaningful to describe," which is different, and better, than simply omitting `alt` entirely (which typically causes screen readers to read out the entire file name instead, which is much less useful).

Lists — `<ul>`, `<ol>`, and `<li>`

Two kinds of lists, and one shared building block:

```
<ul>
    <li>Milk</li>
    <li>Eggs</li>
    <li>Bread</li>
</ul>

<ol>
    <li>Preheat the oven</li>
    <li>Mix the ingredients</li>
    <li>Bake for 20 minutes</li>
</ol>
```

`<ul>` is an **unordered list** (bullet points — use it when the sequence of items genuinely doesn't matter, like a shopping list). `<ol>` is an **ordered list** (automatically numbered — use it when order genuinely matters, like steps in a recipe). Every individual item, in either kind of list, is an `<li>` ("list item") — and every `<li>` must live directly inside a `<ul>` or `<ol>`, never on its own.

Attributes, in more depth

You met the general idea of an attribute on Day 0 (`name="value"`, written inside an opening tag). A few attributes are so common they're worth calling out specifically, since you'll use them across many different elements:

- `id` — gives one specific element a unique name, which must not be repeated anywhere else on the same page. Useful for linking to a specific spot (`<a href="#section2">` jumping to `<h2 id="section2">`) and, as you'll see from Day 3 onward, for targeting one specific element with CSS.
- `class` — gives an element (or, unlike `id`, potentially *many* elements) a label you can use to apply the same styling or behavior to a whole group at once. You'll use `class` constantly starting Day 3.
- `title` — adds a small tooltip that appears when a visitor hovers their mouse over the element.

```
<p id="intro" class="highlight" title="This is the introduction">Welcome to my site!</p>
```

Why "semantic" elements matter — a preview of Day 2

You'll notice, starting tomorrow, that HTML offers many elements beyond generic containers — things like `<nav>`, `<header>`, and `<article>` — whose entire purpose is to describe *what a piece of content actually is*, rather than just being an anonymous box you style however you like. The underlying principle, worth internalizing today even before you meet those specific elements tomorrow: **choose the HTML element that correctly describes what the content actually IS, not the one that happens to look right by default.** A screen reader, a search engine, and another developer reading your code later all rely on this correctness — CSS (starting Day 3) is the tool for controlling how something *looks*; HTML's job is to correctly describe what something *is*, first.

Exercises

Open `starter.html`, follow the `<!-- TODO -->` markers, and check your work by reloading the file in your browser and comparing against `CHECKLIST.md`.